

Coding for Artificial Intelligence

Week 5

Problem Set 5

Manual Sequence Traversal and Flow-Control Mastery

Project: The Calibration Log Synthesizer

Course	Coding for AI (Week 5)
Author	Dr. Prateek, Assistant Professor, School of AI
Format	In-class project, 1 hour 30 minutes total, inclusive of write-up and submission
Total Marks	100

Read this document in full before you begin writing code.

Contents

1	Objective	2
2	The Problem: The Calibration Log Synthesizer	2
3	Implementation Restrictions	2
4	Manual Backward Traversal	3
5	The Walrus-Controlled Outer Loop	3
6	Functional Requirements	4
7	Worked Trace	4
8	Edge Cases to Handle	5
9	Suggested Time Allocation	5
10	Deliverables	5
	Academic Integrity	6

1 Objective

In Week 4 you built an audit engine using nested loops, `continue`, `break`, and the loop `else` clause. This week's project **deliberately repeats that same logical skeleton** — on purpose. Flow control inside nested loops is one of the most consistently misunderstood areas in introductory computing, and the only reliable way to make it automatic is to rebuild it, from a blank file, more than once.

This time, three constraints are added that force you to reason about *how* Python's built-in tools work, by temporarily taking those tools away:

- (i) you will track sequence length and traverse a list **backwards** using only manual index arithmetic — no `len()`, no negative slicing, no `reversed()`;
- (ii) you will compute sums, maxima, and minima using manual accumulator and comparison logic — no `sum()`, `max()`, or `min()`; and
- (iii) you will drive your outer loop with a **walrus-operator while loop** instead of a plain `for` loop over the dataset.

None of this is idiomatic Python for production code — in real projects you should always prefer `len()`, `sum()`, slicing, and a plain `for` loop, because they are shorter, faster, and far less error-prone. The point here is purely diagnostic: if you can correctly reconstruct what these built-ins do by hand, you have genuinely understood the underlying mechanics, rather than having memorised a function call.

Time budget. This entire activity — coding, testing, and writing up your submission — is designed to fit in **90 minutes**. A suggested breakdown is given in Section 9.

2 The Problem: The Calibration Log Synthesizer

You are given a **Calibration Feed**: a list of lists, where each inner list is a batch of sensor calibration readings. The first element of every non-empty batch is its **Calibration ID** (an integer); the remaining elements are floating-point readings or one of three control signals.

```

1 calibration_feed = [
2     [201, 6.0, 9.5, "IGNORE", 4.0],
3     [],
4     [202, 11.2, "FAULT", 7.8, 5.5],
5     [203, 14.0, 3.5, 8.25],
6     [204, 2.75, "HALT", 6.0]
7 ]

```

Signal	Meaning
"IGNORE"	Noise. Skip this one reading and continue the batch.
"FAULT"	This batch's sensor has failed. Abandon the <i>rest of this batch only</i> and move to the next one.
"HALT"	Catastrophic failure. Abandon this batch <i>and</i> stop processing the feed entirely.

3 Implementation Restrictions

To force manual reasoning over the underlying mechanics, your script, `calibration_synthesizer.py`, must **not** use any of the following. Each is explained so the restriction is a learning exercise,

not an arbitrary rule.

1. **No `len()`.** Calculate the length of any list with a manual counter loop:

```
1 batch_length = 0
2 for _ in batch:
3     batch_length += 1
```

2. **No `sum()`, `max()`, or `min()`.** Use a manual accumulator for sums, and manual comparison loops for the running maximum and minimum (see Section 4).
3. **No `enumerate()` or `zip()`.** Track every index yourself, with a counter variable you increment inside the loop.
4. **No negative-step slicing (`::-1`), no `reversed()`, and no negative step in `range()`.** You must still process each batch's readings from **right to left**. Do this with a positive-stepping `for` loop whose index you *mathematically map* onto the reversed position — see Section 4.
5. **No function definitions.** Do not use `def` or `lambda` anywhere. The entire script runs top-down in the global scope. (This restriction exists purely to keep the whole data flow visible in one place while you are learning it; once these patterns are second nature, breaking logic into functions is the *correct* thing to do, and you are encouraged to do so in every other assignment.)

4 Manual Backward Traversal

This is the trickiest part of the assignment, so work through it on paper before you type anything.

For a batch of length L (including the Calibration ID at index 0), you need to visit indices $L-1, L-2, \dots, 1$ — every reading, from the last one back to (but not including) the ID. Write a loop where i counts **upward** in the ordinary way, and compute the actual index you want from i :

```
1 for i in range(1, batch_length):
2     target_index = batch_length - i
3     reading = batch[target_index]
4     ...
```

Trace this by hand for a batch of length 5: as i goes 1, 2, 3, 4, `target_index` goes 4, 3, 2, 1 — the last reading first, the first reading (after the ID) last. This is the core trick behind the whole assignment; everything else follows from it.

5 The Walrus-Controlled Outer Loop

Drive your outer iteration with a `while` loop whose condition re-evaluates a one-item slice of the feed using the walrus operator:

```
1 feed_cursor = 0
2
3 while (current_slice := calibration_feed[feed_cursor:feed_cursor+1]):
4     batch = current_slice[0]
5     ...
6     feed_cursor += 1
```

Why this works. Slicing past the end of a list in Python never raises an error — it simply returns an empty list. So once `feed_cursor` reaches the end of `calibration_feed`, `calibration_feed[feed_cursor:feed_cursor+1]` evaluates to `[]`, which is falsy, and the `while` loop condition becomes `False` on its own — no separate length check is needed to know when to stop.

6 Functional Requirements

Your script must accomplish the following, in order, using a single nested loop structure:

1. **Empty batch check.** If a batch is empty, print "Batch [Index] is EMPTY. Proceeding." and move on immediately. Use Python's implicit Boolean evaluation (`if not batch:`) — this is legal even though `len()` itself is banned, because you are testing truthiness, not calling a length function.
2. **Reverse reading traversal.** For non-empty batches, isolate the Calibration ID at index 0, then process the remaining readings right to left using the mapping from Section 4.
3. **Control signal handling.**
 - (a) "IGNORE": print "Signal IGNORE encountered at Batch [ID]." and `continue`.
 - (b) "FAULT": print "Signal FAULT detected. Suppressing batch [ID]." and `break` out of the inner loop.
 - (c) "HALT": print "Signal HALT detected. Executing emergency protocol.", set a flag recording that the entire feed must stop, and `break` out of the inner loop. (As in Week 4, a single `break` only exits the inner loop — check your flag after the inner loop ends, and `break` the outer loop too when it is set.)
4. **Metric generation.** For every valid floating-point reading: add it to a batch-specific `batch_sum`; and update running `global_max` and `global_min` variables with manual comparisons (no `max()/min()`).
5. **Loop-else integrity check.** Attach an `else` block to the **inner** loop. It must run only if that batch's inner loop completed without hitting "FAULT" or "HALT". Inside it:
 - (a) if the Calibration ID is even, multiply `batch_sum` by 1.5; if odd, by 0.8;
 - (b) add the result to a running `total_checksum`.

7 Worked Trace

Running a correct implementation against the dataset in Section 2 produces the following, verified output:

```

1 Evaluating Batch 0 (ID: 201)...
2 Signal IGNORE encountered at Batch 201.
3 Batch 1 is EMPTY. Proceeding.
4 Evaluating Batch 2 (ID: 202)...
5 Signal FAULT detected. Suppressing batch 202.
6 Evaluating Batch 3 (ID: 203)...
7 Evaluating Batch 4 (ID: 204)...
8 Signal HALT detected. Executing emergency protocol.
9
10 =====
11 CALIBRATION COMPLETE: EMERGENCY TERMINATION
```

```

12 =====
13 Total Valid Readings Processed: 9
14 Global Calibration Checksum: 36.2
15 Maximum Reading Encountered: 14.0
16 Minimum Reading Encountered: 3.5

```

How to check your own arithmetic. Batch 0 (ID 201, odd) sums its three valid readings ($4.0 + 9.5 + 6.0 = 19.5$) and applies the odd multiplier: $19.5 \times 0.8 = 15.6$. Batch 2 hits "FAULT" before completing, so its `else` block never runs — its two valid readings (5.5 and 7.8) still count toward `Total Valid Readings`, but its sum never reaches the checksum. Batch 3 (ID 203, odd) completes fully: $8.25 + 3.5 + 14.0 = 25.75$, and $25.75 \times 0.8 = 20.6$. The running checksum is therefore $15.6 + 20.6 = 36.2$. Batch 4 processes one valid reading (6.0) before "HALT" fires, so that reading counts toward the total but — exactly like Batch 2 — its `else` block is skipped, and it contributes nothing further to the checksum.

8 Edge Cases to Handle

1. **An empty batch in the middle of the feed** (as in Section 2) must not disturb your `feed_cursor` arithmetic or cause the walrus-controlled loop to misbehave.
2. **"HALT" as the very first reading** processed in a batch (i.e., the last element of the list). Confirm your emergency flag still fires correctly even though no numeric readings were processed yet in that batch.
3. **A batch containing only the Calibration ID and no readings**, e.g., [205]. Your `range(1, batch_length)` should simply produce no iterations at all — confirm this does not error, and that the (empty) inner loop still completes normally, running its `else` block with a `batch_sum` of 0.0.

9 Suggested Time Allocation

Phase	Description	Duration
Implementation	Manual traversal, metrics, and flow control, using the worked trace to self-check	55 minutes
Testing	Edge cases from Section 8	20 minutes
Documentation & Submission	Header comment, final write-up	15 minutes
Total		90 minutes

10 Deliverables

1. **Source Code:** A single file named `calibration_synthesizer.py`, containing no `def`/`lambda` and none of the banned built-ins from Section 3.
2. **Header Documentation:** A comment block at the top of your file explaining, in your own words, the index-mapping formula from Section 4 and why the walrus-controlled `while` loop naturally stops at the end of the feed without a separate length check.
3. **Submission Note:** A short paragraph (100–150 words) identifying which batch(es) in the

worked trace had their `else` block skipped, and why, in terms of `break` versus normal loop completion.

Academic Integrity

You may discuss approaches with peers during the supervised project session, but the code and documentation you submit must be written independently and must reflect your own understanding. Submissions found to be substantially copied will be treated in accordance with the institute's academic integrity policy.